Guion para responder

Cómo funciona el carrito y cómo se evita la mezcla

Texto para leer en voz alta, en orden, con el código que se muestra después de cada frase.

Autor	**Elías Caranza**
Fecha	**26 de octubre de 2026**
Formato	**Guion + código real con líneas**
Pregunta	**Flujo del carrito y bloqueo de mezcla**

CÓMO USARLO

Leé solo los recuadros DECIR. Después de cada uno abrí el archivo y las líneas del recuadro MOSTRAR. El texto ya dice qué hace cada línea y a qué parte llama; no hay que improvisar.

ORDEN DEL GUION

1. Cómo funciona: botón, MenuScreen, addToCart, agregarProducto, CartContext, CartScreen y totales. 2. Cómo se evita: los cinco puntos de la cafetería. 3. Cantidades, eliminar y vaciar. 4. Cambio de cafetería. 5. Del carrito a los items del pedido.

ANEXOS

El diccionario y el mapa de archivos del final son de consulta. No se leen en voz alta.

1. Cómo funciona el carrito: de MenuScreen.js a CartScreen.js

DECIR

Para responder cómo funciona, empiezo por el botón. Abro ProductCard.js:43-46, que es el componente de la tarjeta. La línea 44 no deja seguir si el producto no está disponible o si ya se está agregando; la línea 45 hace la llamada onAdd?.(product). Ese onAdd es el que sigue en MenuScreen.

MOSTRAR · ProductCard.js 43-46 — el botón llama a onAdd

```
43  const handleAdd = () => {
44    if (!available || adding) return;
45    onAdd?.(product);
46  };
```

DECIR

Ese onAdd llega a MenuScreen.js:303-321, que es la pantalla del menú. En 308-309 se revisan dos cosas antes de seguir: si el producto no está disponible, termina sin agregar; si su cafeteriaId no coincide con la cafetería abierta, tampoco sigue. En 313-314 prende addingId 250 milisegundos para el feedback, y en 318 llama a onAddToCart con el producto y la cafetería.

MOSTRAR · MenuScreen.js 303-321 — validación y llamada al carrito

```
303  const [addingId, setAddingId] = useState(null);
304
305  const handleAdd = (product) => {
306    // Cinturón de seguridad también del lado del carrito: nunca agregar
307    // un producto que no sea de esta cafetería o que esté agotado.
308    if (!product?.available) return;
309    if (Number(product.cafeteriaId) !== Number(cafeteriaId)) return;
310    // Pequeña transición/feedback de carga moderado al agregar desde catálogo
311    const pid = product?.id;
312    if (pid != null) {
313      setAddingId(pid);
314      setTimeout(() => setAddingId(null), 250);
315    }
316    // Se pasa la cafetería para que el carrito sepa a qué pedido pertenece
317    // cada ítem y no mezcle productos de dos menus distintos.
318    onAddToCart?.(product, cafeteria);
319    // Si venía desde el detalle, se cierra para volver al menú
320    setProductoDetalle(null);
321  };
```

DECIR

Esa llamada llega a App.tsx:366-400, a la función addToCart, que vuelve a validar: 367-369 se detiene si no está disponible; 371 normaliza el id; 378 compara la cafetería del carrito con la del producto. Si no coinciden, muestra la alerta de 379-395 y no agrega. Si coinciden, en 399 llama a agregarProducto.

MOSTRAR · App.tsx 366-400 — addToCart y la regla de una sola cafetería

```
366  const addToCart = (product: Product, cafeteria: Cafeteria) => {
367    if (!product.available) {
368      return;
369    }
370
371    const cafeteriaId = Number(cafeteria.id);
372
373    // El carrito ya tiene productos de otra cafetería: no se mezclan.
374    // La RPC recibe un único `p_cafeteria_id`, así que mezclar guardaba los
375    // precios bien pero mandaba los productos de A con el id de B. Antes pasaba
376    // en silencio. Ahora se le explica al usuario y se le da la opción de
377    // cambiar de cafetería.
378    if (cartCafeteria && cartCafeteria.id !== cafeteriaId) {
379      Alert.alert(
380        'Tu carrito es de otra cafetería',
381        `Ya tenés productos de ${cartCafeteria.nombre}. Un pedido es de una sola cafetería. Si querés pe
     dir en ${cafeteria.nombre}, vaciá el carrito.`,
382        [
383          { text: 'Cancelar', style: 'cancel' },
384          {
385            text: 'Vaciar y agregar',
386            style: 'destructive',
387            // Se vacía y se agrega en el mismo toque: si solo se vaciaba, el
388            // botón prometía algo que no pasaba.
389            onPress: () => {
390              clearCart();
391              agregarProducto(product, cafeteria);
392            },
393          },
394        ]
395      );
396      return;
397    }
398
399    agregarProducto(product, cafeteria);
400  };
```

DECIR

agregarProducto, la función de App.tsx:340-364, usa setCart con la lista más reciente en 343. En 344 busca si el producto ya está; si está, 346-348 le suma 1 a quantity. Si no está, 350-362 crea el CartItem con price, quantity: 1 y cafeteriaId; por eso el ítem conserva esos datos.

MOSTRAR · App.tsx 340-364 — sumar cantidad o crear el ítem

```
340   const agregarProducto = (product: Product, cafeteria: Cafeteria) => {
341     const cafeteriaId = Number(cafeteria.id);
342
343     setCart((currentCart: CartItem[]) => {
344       const existing = currentCart.find((item) => item.id === product.id);
345       if (existing) {
346         return currentCart.map((item) =>
347           item.id === product.id ? { ...item, quantity: item.quantity + 1 } : item
348         );
349       }
350       return [
351         ...currentCart,
352         {
353           id: product.id,
354           name: product.name,
355           description: product.description,
356           price: product.price,
357           emoji: product.emoji,
358           quantity: 1,
359           cafeteriaId,
360           cafeteriaNombre: cafeteria.nombre,
361         },
362       ];
363     });
364   };
```

DECIR · QUÉ ES UN CONTEXTO

Un contexto de React es un valor que se declara una vez y queda disponible para todas las pantallas que están dentro de su Provider, que es el componente que entrega ese valor. No hay que pasarlo de pantalla en pantalla: cada pantalla lo pide con useContext y recibe el mismo valor. Acá ese valor compartido es el carrito.

DECIR

El arreglo queda en CartContext.js:14. En 6-11 se crea CartContext con createContext y un valor inicial vacío: cart [], setCart, clearCart y hydrated false. Ese false de 10 es el valor por defecto del contexto; el estado real es el de 18, que también empieza false porque en el primer render todavía no se leyó AsyncStorage. En 13 empieza CartProvider, que es el componente que envuelve la app; en 14 está el estado real cart; y en 18, hydrated.

MOSTRAR · CartContext.js 6-18 — se crea el contexto y el estado real

```
6   const CartContext = createContext({
7     cart: [],
8     setCart: () => {},
9     clearCart: () => {},
10    hydrated: false,
11  });
12
13  export const CartProvider = ({ children }) => {
14    const [cart, setCart] = useState([]);
15    // `hydrated` evita pintar un carrito vacío antes de terminar de leer
16    // AsyncStorage: sin él, el primer render siempre mostraba el carrito vacío
17    // y pisaba lo guardado en disco.
18    const [hydrated, setHydrated] = useState(false);
```

DECIR · QUÉ ES UN EFECTO

Un efecto es un bloque que React ejecuta después de mostrar el componente en pantalla; no es una función que se llama a mano. La lista del final son las dependencias: si está vacía, se ejecuta una sola vez; si tiene valores, se ejecuta cuando esos valores cambian.

DECIR

hydrated, en 18, empieza en false. El efecto 21-41 termina con [] en 41, así que se ejecuta una sola vez, cuando el componente aparece. Dentro, en 25 lee AsyncStorage, en 29 carga el carrito guardado con setCart y en 35 marca hydrated en true. El efecto 45-50 termina con [cart, hydrated] en 50, así que se ejecuta cada vez que cambia el carrito o hydrated. En 46 no escribe hasta que hydrated sea true; en 47 escribe en AsyncStorage. Eso evita que el primer render vacío pise lo guardado; por eso el carrito sobrevive al cierre.

MOSTRAR · CartContext.js 21-41 — efecto de lectura; [] en 41

```
21  useEffect(() => {
22    let cancelado = false;
23    (async () => {
24      try {
25        const crudo = await AsyncStorage.getItem(CART_STORAGE_KEY);
26        if (!cancelado && crudo) {
27          const guardado = JSON.parse(crudo);
28          if (Array.isArray(guardado)) {
29            setCart(guardado);
30          }
31        }
32      } catch (err) {
33        console.warn('[CartContext] No se pudo recuperar el carrito:', err);
34      } finally {
35        if (!cancelado) setHydrated(true);
36      }
37    })();
38    return () => {
39      cancelado = true;
40    };
41  }, []);
```

MOSTRAR · CartContext.js 45-51 — efecto de escritura; dependencias en 50

```
45  useEffect(() => {
46    if (!hydrated) return;
47    AsyncStorage.setItem(CART_STORAGE_KEY, JSON.stringify(cart)).catch((err) =>
48      console.warn('[CartContext] No se pudo guardar el carrito:', err)
49    );
50  }, [cart, hydrated]);
```

DECIR · QUÉ ES EL PROVIDER

El Provider es el componente que entrega el valor del contexto a los componentes que están dentro. En 13, CartProvider arma el estado; en 55, CartContext.Provider recibe en value el carrito y las funciones; en 56, children es todo lo que envuelve. Cualquier pantalla dentro de children lo lee con useCart, que en 61 es useContext(CartContext). Si no estuviera el Provider, useContext devolvería el valor por defecto de 6-11, que es un carrito vacío.

MOSTRAR · CartContext.js 55-61 — el Provider reparte el carrito y useCart lo lee

```
55      <CartContext.Provider value={{ cart, setCart, clearCart, hydrated }}>
56        {children}
57      </CartContext.Provider>
58    );
59  };
60
61  export const useCart = () => useContext(CartContext);
```

DECIR · EN UNA FRASE

En una frase: el Provider reparte el carrito a las pantallas; los efectos lo cargan al inicio y lo guardan cuando cambia.

DECIR

CartScreen.js:149-156, que es la pantalla del carrito, recibe el arreglo y renderiza un CartItem por ítem. CartItem es el componente de CartItem.js:10-16: recibe item y los callbacks, pinta la fila y no toca el arreglo. El tipo del objeto es otro: CartItem de App.tsx:42-55, que contiene id, name, description, price, emoji, quantity, cafeteriaId y cafeteriaNombre.

MOSTRAR · CartScreen.js 149-156 — un CartItem por ítem

```
149           renderItem={({ item }) => (
150             <CartItem
151               item={item}
152               onIncrease={onIncrease}
153               onDecrease={onDecrease}
154               onRemove={onRemove}
155             />
156           )}
```

MOSTRAR · CartItem.js 10-24 — props, subtotal y mínimo

```
10  const CartItem = ({
11    item,
12    onIncrease,
13    onDecrease,
14    onRemove,
15  }) => {
16    const [showDeleteModal, setShowDeleteModal] = useState(false);
17
18    const subtotal = item.price * item.quantity;
19    const isMinimumQuantity = item.quantity === 1;
20
21    const handleRemove = () => {
22      setShowDeleteModal(false);
23      onRemove(item.id);
24    };
```

MOSTRAR · App.tsx 42-55 — el tipo del ítem

```
42 type CartItem = {
43   id: number;
44   name: string;
45   description: string;
46   price: number;
47   emoji: string;
48   quantity: number;
49   /**
50    * De qué cafetería es el ítem. El pedido se hace contra UNA cafetería, así
51    * que esto es lo que impide mezclar productos de dos menus distintos.
52    */
53   cafeteriaId: number;
54   cafeteriaNombre: string;
55 };
```

DECIR

useCartTotals es el hook de useCartTotals.js:4-10: en 5 suma las cantidades, en 6 suma price × quantity y en 9 deja total igual a subtotal. CartScreen lo usa en 49 y muestra los valores en 176 y 179; App.tsx:695-697 usa totalProducts para el badge.

MOSTRAR · useCartTotals.js 4-10 — totalProducts, subtotal y total

```
4  export const useCartTotals = (cart = []) => {
5    const totalProducts = cart.reduce((sum, item) => sum + (item.quantity || 0), 0);
6    const subtotal = cart.reduce((sum, item) => sum + item.price * item.quantity, 0);
7    const total = subtotal;
8
9    return { totalProducts, subtotal, total };
10 };
```

MOSTRAR · CartScreen.js 173-181 — el resumen usa el hook

```
173                <Card padding="lg" style={styles.summaryCard}>
174                  <View style={styles.summaryRow}>
175                    <Text style={styles.subtotalLabel}>
176                      Subtotal ({totalProducts} productos)
177                    </Text>
178
179                    <Text style={styles.subtotal}>
180                      ${subtotal.toLocaleString('es-CL')}
181                    </Text>
```

DECIR

Con eso queda la primera parte: botón, MenuScreen, addToCart, agregarProducto, CartContext, CartScreen y totales. Ahora paso a la segunda parte: cómo se evita que un estudiante mezcle cafeterías.

2. Cómo se evita que un estudiante mezcle cafeterías

DECIR

Para responder cómo se evita, la metodología es una sola cafetería por pedido, y se ve en cinco puntos. Primero, el menú solo consulta productos de la cafetería abierta: MenuScreen.js:175-192 filtra por cafeteria_id y descarta cualquier fila cuyo cafeteriaId no coincida. Por eso la pantalla no ofrece productos de otra cafetería.

MOSTRAR · MenuScreen.js 175-192 — solo productos de la cafetería

```
175         const { data, error: queryError } = await supabase
176           .from('productos')
177           .select('id, cafeteria_id, categoria_id, nombre, descripcion, precio, imagen_url, stock, stock
    _minimo, activo, eliminado_en, categorias(nombre)')
178           .eq('cafeteria_id', cafeteriaId)
179           .eq('activo', true)
180           .is('eliminado_en', null);
181
182         if (queryError) throw queryError;
183
184         // Cinturón de seguridad: aunque la consulta ya filtra, descartamos
185         // cualquier fila que venga de otra cafetería. Así el requisito "no
186         // mostrar productos de otras cafeterías" se cumple por partida doble.
187         const propios = (data || []).filter(
188           (row) => Number(row.cafeteria_id) === Number(cafeteriaId)
189         );
190
191         // productsMap mantiene el id numérico, que es lo que necesita el carrito
192         setProducts(propios.map(mapProduct).filter((p) => Number.isFinite(p.id)));
```

DECIR

Segundo, al agregar se hace una revisión en MenuScreen.js:308-309: si no está disponible o el cafeteriaId no coincide, no llama al carrito. Esa revisión impide agregar desde una tarjeta que no corresponde.

MOSTRAR · MenuScreen.js 308-309 — revisión antes de llamar al carrito

```
308     if (!product?.available) return;
309     if (Number(product.cafeteriaId) !== Number(cafeteriaId)) return;
```

DECIR

Tercero, el carrito registra de qué cafetería es cada ítem. El tipo CartItem de App.tsx:42-55 tiene cafeteriaId y cafeteriaNombre; agregarProducto los copia en 359-360. La cafetería del pedido no se lee de la pantalla: cartCafeteria es el objeto derivado en App.tsx:336-337 y se calcula desde el primer ítem con cart[0].cafeteriaId.

MOSTRAR · App.tsx 336-337 — cartCafeteria sale del primer ítem

```
336   const cartCafeteria: { id: number; nombre: string } | null =
337     cart.length > 0 ? { id: cart[0].cafeteriaId, nombre: cart[0].cafeteriaNombre } : null;
```

DECIR

Cuarto, addToCart compara esa cafetería con la del producto nuevo en App.tsx:378. Si el carrito ya tiene otra, no agrega: muestra la alerta 379-395. Cancelar en 383 no cambia nada; Vaciar y agregar en 385-392 ejecuta clearCart en 390 y agregarProducto en 391; el return 396 evita que siga al agregado normal.

MOSTRAR · App.tsx 378-399 — alerta y decisión

```
378     if (cartCafeteria && cartCafeteria.id !== cafeteriaId) {
379       Alert.alert(
380         'Tu carrito es de otra cafetería',
381         `Ya tenés productos de ${cartCafeteria.nombre}. Un pedido es de una sola cafetería. Si querés pe
    dir en ${cafeteria.nombre}, vaciá el carrito.`,
382         [
383           { text: 'Cancelar', style: 'cancel' },
384           {
385             text: 'Vaciar y agregar',
386             style: 'destructive',
387             // Se vacía y se agrega en el mismo toque: si solo se vaciaba, el
388             // botón prometía algo que no pasaba.
389             onPress: () => {
390               clearCart();
391               agregarProducto(product, cafeteria);
392             },
393           },
394         ]
395       );
396       return;
397     }
398
399     agregarProducto(product, cafeteria);
```

DECIR

Quinto, el backend vuelve a validar. En pedidos.service.js:267-272, si un producto no pertenece a la cafetería del pedido, responde 400. Y la RPC recibe un solo p_cafeteria_id en rpc_qr.sql:21-23, así que no existe un pedido con dos cafeterías.

MOSTRAR · pedidos.service.js 262-272 — pertenencia cruzada

```
262   // Regla de negocio: TODOS los productos del pedido deben pertenecer a la
263   // cafeteria indicada por cafeteria_id. ms-menus ya garantiza que existen,
264   // estan activos y no borrados (si no, responde 404); aqui se comprueba la
265   // pertenencia cruzada, que ms-menus no puede validar porque /precios no recibe
266   // cafeteria_id.
267   const productoDeOtraCafeteria = productos.find(
268     (p) => Number(p.cafeteria_id) !== cafeteriaId
269   );
270   if (productoDeOtraCafeteria) {
271     throw new HttpError(400, 'Uno o mas productos no pertenecen a la cafeteria indicada');
272   }
```

MOSTRAR · rpc_qr.sql 21-25 — un solo p_cafeteria_id

```
21  CREATE OR REPLACE FUNCTION public.procesar_pago(
22    p_cafeteria_id  BIGINT,
23    p_items         JSONB,
24    p_franja_retiro TEXT DEFAULT NULL
25  )
```

DECIR

Con esos cinco puntos, si un estudiante recorre A, vuelve al listado y entra a B, el carrito no acepta el producto de B. Si quiere pedir en B, tiene que vaciar y agregar; el pedido queda con una sola cafetería. Esa es la metodología que evita la mezcla.

3. Qué pasa al aumentar, disminuir, eliminar o vaciar

DECIR

Si preguntan por las cantidades, los controles están en CartItem.js:55-85. El menos se deshabilita cuando quantity es 1 (18-19); el más sigue activo. Ambos llaman a App.tsx:402-418: increaseQuantity suma 1 en 405; decreaseQuantity resta 1 en 414 y filtra quantity > 0 en 416; si llega a 0, el ítem desaparece.

MOSTRAR · CartItem.js 55-85 — menos deshabilitado en 1, más activo

```
55              <TouchableOpacity
56                style={[
57                  styles.quantityButton,
58                  isMinimumQuantity && styles.quantityButtonDisabled,
59                ]}
60                onPress={() => onDecrease(item.id)}
61                disabled={isMinimumQuantity}
62              >
63                <Text
64                  style={[
65                    styles.quantityButtonText,
66                    isMinimumQuantity &&
67                      styles.quantityButtonTextDisabled,
68                  ]}
69                >
70                  –
71                </Text>
72              </TouchableOpacity>
73
74              <Text style={styles.quantity}>
75                {item.quantity}
76              </Text>
77
78              <TouchableOpacity
79                style={styles.quantityButton}
80                onPress={() => onIncrease(item.id)}
81              >
82                <Text style={styles.quantityButtonText}>
83                  +
84                </Text>
85              </TouchableOpacity>
```

MOSTRAR · App.tsx 402-408 — aumentar cantidad

```
402   const increaseQuantity = (id: number) => {
403     setCart((currentCart: CartItem[]) =>
404       currentCart.map((item) =>
405         item.id === id ? { ...item, quantity: item.quantity + 1 } : item
406       )
407     );
408   };
```

MOSTRAR · App.tsx 410-424 — disminuir y eliminar

```
410  const decreaseQuantity = (id: number) => {
411    setCart((currentCart: CartItem[]) =>
412      currentCart
413        .map((item) =>
414          item.id === id ? { ...item, quantity: item.quantity - 1 } : item
415        )
416        .filter((item) => item.quantity > 0)
417    );
418  };
419
420  const removeFromCart = (id: number) => {
421    setCart((currentCart: CartItem[]) =>
422      currentCart.filter((item) => item.id !== id)
423    );
424  };
```

DECIR

Para eliminar, CartItem.js:21-24 cierra el modal y llama a onRemove con el id; el modal está en 88-140. App.tsx:420-424 filtra el arreglo y saca el ítem. Para vaciar, clearCart está en CartContext.js:52 y hace setCart([]); se llama en Vaciar y agregar (App.tsx:385-392), pago aprobado (507) y logout o salir de invitado (288). No hay un botón independiente de vaciar carrito.

MOSTRAR · CartContext.js 52 — clearCart

```
52  const clearCart = useCallback(() => setCart([]), []);
```

MOSTRAR · App.tsx 385-392 — vaciar y agregar

```
385            text: 'Vaciar y agregar',
386            style: 'destructive',
387            // Se vacía y se agrega en el mismo toque: si solo se vaciaba, el
388            // botón prometía algo que no pasaba.
389            onPress: () => {
390              clearCart();
391              agregarProducto(product, cafeteria);
392            },
```

DECIR

Al cambiar de cafetería, el carrito no se borra solo por navegar. selectedCafeteria es la cafetería abierta del menú; la del pedido es cart[0] en App.tsx:336-337. Si el carrito está vacío, el producto de la nueva entra en 399; si tiene ítems, la alerta de 378-396 ofrece Cancelar o Vaciar y agregar. Eso ya quedó en la segunda parte.

4. Del carrito a los items del pedido

DECIR

Para responder qué información del carrito se transforma en items: al pagar, App.tsx:468-475 arma el body. En 469 manda cafeteria_id; en 470-473 recorre cart y manda producto_id en 471 y cantidad en 472; en 474 manda franja_retiro. No manda name, price, subtotal ni total.

MOSTRAR · App.tsx 468-475 — el body que sale de la app

```
468       const cuerpo = {
469         cafeteria_id: cafeteriaId,
470         items: cart.map((item) => ({
471           producto_id: item.id,
472           cantidad: item.quantity,
473         })),
474         franja_retiro: franjaRetiro.franja,
475       };
```

DECIR

Ese body llega a pedidos.service.js:281-285, que llama a procesar_pago con p_cafeteria_id, p_items y p_franja_retiro. procesar_pago, la RPC de rpc_qr.sql:21, recorre los items en 65-87, lee productos.precio en 77-79, calcula subtotal en 86 y acumula total en 87.

MOSTRAR · pedidos.service.js 281-285 — llamada a procesar_pago

```
281     const { data: rpcData, error: rpcError } = await cliente.rpc('procesar_pago', {
282       p_cafeteria_id: cafeteriaId,
283       p_items: items,
284       p_franja_retiro: franjaRetiro
285     });
```

MOSTRAR · rpc_qr.sql 65-87 — recorrido, precio real y total

```
65    FOR v_item IN SELECT * FROM jsonb_array_elements(p_items) LOOP
66      v_producto_id := (v_item ->> 'producto_id')::BIGINT;
67      v_cantidad    := (v_item ->> 'cantidad')::INTEGER;
68
69      IF v_producto_id IS NULL OR v_cantidad IS NULL OR v_cantidad <= 0 THEN
70        RETURN jsonb_build_object(
71          'ok', false,
72          'motivo', 'item_invalido',
73          'mensaje', 'El carrito tiene un producto invalido.'
74        );
75      END IF;
76
77      SELECT precio INTO v_precio
78      FROM public.productos
79      WHERE id = v_producto_id AND activo = true AND eliminado_en IS NULL;
80
81      IF v_precio IS NULL THEN
82        v_faltantes := v_faltantes || v_producto_id || ',';
83        CONTINUE;
84      END IF;
85
86      v_subtotal := v_precio * v_cantidad;
87      v_total := v_total + round(v_subtotal)::INTEGER;
```

DECIR

Después arma el detalle en rpc_qr.sql:89-94 con producto_id, cantidad, precio_unitario y subtotal. En una frase: el carrito aporta ids y cantidades; la base aporta el precio y el total. Si el precio del menú cambia, el pedido cobra el precio real de productos.

MOSTRAR · rpc_qr.sql 89-94 — detalle con precio y subtotal

```
89      v_detalles := v_detalles || jsonb_build_array(jsonb_build_object(
90        'producto_id',     v_producto_id,
91        'cantidad',        v_cantidad,
92        'precio_unitario', v_precio,
93        'subtotal',        v_subtotal
94      ));
```

DECIR

Con esto puedo responder las dos partes: cómo funciona el flujo y cómo se evita la mezcla. Si me piden una sola frase: el carrito es un arreglo de CartItem con cafeteriaId; agregar compara esa cafetería, y el pedido manda un solo cafeteria_id que el backend y la RPC vuelven a validar.

DECIR · CIERRE CORTO

Si me apuran, cierro con tres frases: el carrito es estado compartido en CartContext; la regla de una sola cafetería está en MenuScreen, en App y en el backend; y el cliente manda ids y cantidades mientras la base pone el precio y el total.

9. Diccionario de funciones y APIs

En sentido estricto, solo la primera tabla son funciones y construcciones nativas del lenguaje JavaScript/TypeScript. Las demás son APIs de React, React Native, Supabase, fetch, Express y PostgreSQL. Las incluyo porque aparecen en las líneas que mostrás y cumplen el mismo papel práctico: llamar a algo que ya existe. La columna «Dónde» apunta al archivo y la línea para que puedas saltar al código durante la reunión.

9.1. JavaScript y TypeScript · nativas y globales

Nombre	Tipo	Qué hace	Dónde aparece
`map()`	Array	Recorre el arreglo y devuelve uno nuevo con el resultado de la función.	App 346, 470; MenuScreen 192
`filter()`	Array	Devuelve un arreglo nuevo solo con los elementos que cumplen la condición.	App 416, 422; MenuScreen 187, 192
`find()`	Array	Devuelve el primer elemento que cumple la condición, o undefined.	App 344
`reduce()`	Array	Acumula un valor recorriendo el arreglo; recibe acumulador y elemento.	useCartTotals 5-6; MenuScreen 99
`Array.isArray()`	Array	Devuelve true si el valor es un arreglo.	CartContext 28
`Number()`	Global	Convierte un valor a número; si no se puede, devuelve NaN.	MenuScreen 67, 75; App 371
`Number.isFinite()`	Number	Devuelve true solo si el valor es un número finito (no NaN, no Infinity).	MenuScreen 67, 83, 87, 192
`JSON.parse()`	JSON	Convierte un string JSON en objeto o arreglo.	CartContext 27
`JSON.stringify()`	JSON	Convierte un objeto o arreglo en string JSON.	CartContext 47; backendClient 150
`Boolean()`	Global	Convierte un valor a true o false.	MenuScreen 69
`String()`	Global	Convierte un valor a string.	pedidos.service 220
`setTimeout()`	Global	Ejecuta una función después de N milisegundos.	MenuScreen 314
`toLocaleString()`	Number	Formatea un número según el locale; acá se usa 'es-CL'.	ProductCard 126; CartScreen 180, 198
`trim()`	String	Quita espacios al inicio y al final.	pedidos.service 220
`console.warn()`	Console	Escribe una advertencia en la consola; no detiene la ejecución.	CartContext 33, 48; MenuScreen 194
`async / await`	Sintaxis	Espera una promesa sin bloquear el hilo; el await devuelve el valor resuelto.	backendClient 136; CartContext 23

9.2. React y React Native · hooks y APIs

Nombre	Tipo	Qué hace	Dónde aparece
`createContext()`	React	Crea un contexto con un valor por defecto.	CartContext 6
`useState()`	Hook	Devuelve [valor, setter]; el setter re-renderiza el componente.	CartContext 14, 18; CartScreen 48
`useEffect()`	Hook	Ejecuta una función después del render; puede devolver una limpieza.	CartContext 21, 45; Comandas 124
`useCallback()`	Hook	Memoriza una función para que no cambie de identidad entre renders.	CartContext 52; Comandas 110
`useContext()`	Hook	Lee el valor actual de un contexto.	CartContext 61

Nombre	Tipo	Qué hace	Dónde aparece
`useMemo()`	Hook	Memoriza un valor calculado y lo recalcula solo si cambian sus dependencias.	MenuScreen 223, 238, 250, 272
`Alert.alert()`	React Native	Muestra un diálogo nativo con título, mensaje y botones.	App 379, 434, 439, 444, 452, 491
`AsyncStorage.getItem()`	AsyncStorage	Lee un string guardado en el almacenamiento persistente.	CartContext 25
`AsyncStorage.setItem()`	AsyncStorage	Guarda un string en el almacenamiento persistente.	CartContext 47
`StyleSheet.create()`	React Native	Registra y valida estilos una sola vez.	ProductCard, CartScreen, CartItem
`FlatList`	Componente	Lista virtualizada; renderiza solo los ítems visibles.	CartScreen 84; MenuScreen
`TouchableOpacity`	Componente	Contenedor presionable con opacidad al tocar.	ProductCard 128; CartItem 55
`Modal`	Componente	Capa modal sobre la pantalla; se controla con visible.	CartItem 99-140
`BackHandler.addEventListener()`	React Native	Escucha el botón físico de atrás en Android.	MenuScreen 290

9.3. Supabase y fetch · acceso a datos

Nombre	Tipo	Qué hace	Dónde aparece
`supabase.from()`	Supabase	Abre una consulta sobre una tabla.	MenuScreen 175
`.select()`	Supabase	Elige columnas y relaciones que se devuelven.	MenuScreen 177
`.eq()`	Supabase	Agrega una condición de igualdad al WHERE.	MenuScreen 178-179
`.is()`	Supabase	Compara con NULL (IS NULL / IS NOT NULL).	MenuScreen 180
`supabase.rpc()`	Supabase	Llama a una función SQL y devuelve su resultado.	pedidos.service 281
`fetch()`	Web API	Hace una petición HTTP y devuelve una promesa con Response.	backendClient 142
`Response.text()`	Web API	Lee el cuerpo de la respuesta como texto.	backendClient 157
`Response.ok`	Web API	true si el status HTTP está entre 200 y 299.	backendClient 160

9.4. PostgreSQL y PL/pgSQL · lo que corre en la RPC

Nombre	Tipo	Qué hace	Dónde aparece
`jsonb_array_length()`	JSONB	Devuelve la cantidad de elementos de un arreglo JSONB.	rpc_qr.sql 57
`jsonb_array_elements()`	JSONB	Expande un arreglo JSONB en filas.	rpc_qr.sql 65, 172
`jsonb_build_object()`	JSONB	Construye un objeto JSONB a partir de pares clave/valor.	rpc_qr.sql 58, 70, 89, 192
`jsonb_build_array()`	JSONB	Construye un arreglo JSONB.	rpc_qr.sql 89
`round()`	Numérica	Redondea un número al entero más cercano (o a N decimales).	rpc_qr.sql 87
`gen_random_uuid()`	UUID	Genera un UUID v4 aleatorio.	rpc_qr.sql 154
`lpad()`	Texto	Rellena un texto a la izquierda hasta N caracteres.	rpc_qr.sql 155
`RAISE EXCEPTION`	PL/pgSQL	Aborta la transacción con un mensaje de error.	rpc_qr.sql 52
`FOR UPDATE`	Cláusula	Bloquea las filas seleccionadas hasta el fin de la transacción.	rpc_qr.sql 118
`RETURNING ... INTO`	Cláusula	Devuelve columnas de la fila insertada/actualizada a variables.	rpc_qr.sql 142, 150
`->>`	Operador	Extrae un campo JSON como texto.	rpc_qr.sql 66, 168-171
`||`	Operador	Concatena textos o arreglos.	rpc_qr.sql 82, 155, 190
`CREATE OR REPLACE FUNCTION`	DDL	Crea o reemplaza una función en la base.	rpc_qr.sql 21

9.5. Express y Node · la capa HTTP del microservicio

Nombre	Tipo	Qué hace	Dónde aparece
`router.post()`	Express	Registra un handler para POST en una ruta.	pedidos.routes 22
`req.body`	Express	Objeto con el cuerpo JSON ya parseado.	pedidos.controller 11
`res.status().json()`	Express	Fija el código HTTP y envía un JSON.	pedidos.controller 13
`next()`	Express	Pasa el control al siguiente middleware; con error, al manejador.	pedidos.controller 15
`module.exports`	Node	Expone el router para que otro archivo lo importe.	pedidos.routes 32

9.6. nginx · directivas, no funciones

Nombre	Tipo	Qué hace	Dónde aparece
`location`	Directiva	Agrupa configuración para un prefijo de URL.	gateway 41
`proxy_pass`	Directiva	Reenvía la petición a otro host/puerto.	gateway 42
`proxy_set_header`	Directiva	Agrega o reemplaza un header antes de reenviar.	gateway 43-46

RECUADRO · Cómo leer las firmas

Cuando veas f(x) → y: f es el nombre, x lo que recibe, y lo que devuelve. Cuando veas async, la función devuelve una promesa y hay que await-earla (o encadenar .then). Cuando veas un método con punto (cart.map), lo de la izquierda es el objeto y lo de la derecha la función que se ejecuta sobre él. Los hooks de React (useState, useEffect) no son funciones puras: dependen del orden de llamada y del componente.

10. Anexo: mapa de archivos y líneas

Si te preguntan «¿dónde está?», esta tabla es la respuesta rápida. Todas las líneas corresponden al árbol de trabajo usado para armar este documento.

Archivo	Líneas clave	Qué se muestra
app-mobile/src/components/ProductCard.js	26, 43-46, 128-141	Props, handleAdd, botón y feedback.
app-mobile/src/screens/MenuScreen.js	66-95, 175-192, 303-321	mapProduct, consulta y guardia de cafetería.
app-mobile/App.tsx	42-55, 336-400, 426-517, 540, 695-697, 718	Modelo, anti-mezcla, checkout, gate, badge y Provider.
app-mobile/src/context/CartContext.js	4-18, 21-41, 45-52, 61	Estado, hidratación, persistencia y hook.
app-mobile/src/screens/CartScreen.js	29-57, 71-84, 149-156, 173-208	Props, totales, lista, resumen y pago.
app-mobile/src/components/CartItem.js	10-24, 55-85, 88-140	Subtotal, mínimo 1, controles y modal.
app-mobile/src/hooks/useCartTotals.js	4-10	totalProducts, subtotal y total.
app-mobile/src/services/backendClient.ts	136-168	apiPost, JWT, fetch y ApiError.
ms-pedidos/src/modules/pedidos/ pedidos.routes.js	18-22, 32	POST /api/pedidos con auth.
ms-pedidos/src/modules/pedidos/ pedidos.controller.js	6-17	Handler, 201 y next(e).
ms-pedidos/src/modules/pedidos/ pedidos.service.js	208-292	Validaciones, cotización, pertenencia y RPC.
app-mobile/tabla_mobile/rpc_qr.sql	21-29, 57-95, 114-191	Firma, precios, wallet, pedido y pago.
Microservices/gateway/nginx.conf	41-47	Ruta /api/pedidos a ms-pedidos.
app-desktop/src/pages/empleado/Comandas.jsx	124-131	Suscripción Realtime del KDS.

RECUADRO · Si el emulador falla en vivo

No defiendas la demo: mostrá el código. Las líneas de este documento son la prueba. Si querés reiniciar: adb reverse tcp:8082 tcp:8082, Metro en 8082, app com.coffeefastapp/.MainActivity, login empleado@coffeefast.cl / 123456. Si aparece PGRST200 entre productos y categorias, es la relación de la base local, no el flujo del carrito.

RECUADRO · Tres frases para cerrar si te apuran

1) «El carrito es estado compartido; la regla de una sola cafetería está en la pantalla, en el estado y en el backend.» 2) «El cliente manda ids y cantidades; el precio y el total los pone la base.» 3) «El carrito se vacía solo cuando el pago devuelve ok, no cuando el HTTP es 201.»

Documento de estudio y reunión. Las referencias de línea corresponden al árbol de trabajo del 26-10-2026. Si el código cambia, verificá el número antes de citarlo en vivo.